

CSC343H5 Lecture 7

Database Design (Functional Dependencies)

Naaz Sibia (naaz.sibia@utoronto.ca)

Admin

- ❑ Checkpoint 4 is out now, due on July 26th
- ❑ Midterm on July 22nd
- ❑ Office hours Updates:
 - ❑ **3 hours next week (Friday) from 12-3PM online!**
 - ❑ cancelled this week, available by appointment/email
 - ❑ next week: in-person office hours cancelled

Admin

- ❑ Checkpoint 4 is out now, due on July 26th
- ❑ Midterm on July 22nd
- ❑ Office hours Updates:
 - ❑ 3 hours next week (Friday) from 12-3PM online!
 - ❑ **cancelled this week, available by appointment/email**
 - ❑ **next week: in-person office hours cancelled**

New Material: Functional Dependency Theory!

Introduction: Why Do We Need Design Theory?

- There are always many different schemas for a given set of data.
 - e.g. you could combine or divide tables.
- How do you pick a schema? Which is better? What does "better" mean?

Introduction: Why Do We Need Design Theory?

- Database Design Theory allows us to improve a schema systematically.
 - The general idea:
 - 1. Express constraints on data; and
 - 2. Use these to decompose the relations.
- The goal: reach a schema in normal form, guaranteeing good properties such as no anomalies.
 - The process of converting to a normal form is called *normalization*.

Avoid Redundancy

- This table has redundant data, and that can lead to anomalies.

name	dorm	courseTaken	dept	favCourse
Janeway	Voyager	Databases	CS	Ethics
Janeway	Voyager	Ethics	Philosophy	Ethics
Spock	Enterprise	Databases	CS	Databases

Update anomaly: if Janeway is transferred to a different dorm, will we remember to change each of her tuples?

Deletion anomaly: if nobody takes Ethics, we lose track of the fact that the Philosophy department offers it.

Normalization

- The general idea is to:
 - express constraints on data; and
 - use these to decompose the relations.
- Ultimately, get a schema that is in **normal form**.
 - “Normal” meaning conforming to a standard.
 - “Normal Form” referring to guaranteeing ‘good’ properties; such as no anomalies.

Part I

Functional Dependency Theory



Functional Dependencies (FDs)

Definition

$X \rightarrow Y$ holds in R means: whenever two tuples of R agree on all attributes in X , they must also agree on all attributes in Y .

- X functionally determines Y .
- Conventions: X, Y, Z denote sets of attributes; A, B, C denote single attributes.
 - No braces needed, write ABC rather than $\{A,B,C\}$.
- Why "functional"? There is a mathematical function taking a value for X and giving a unique value for Y .
- Why "dependency"? The value of Y depends on the value of X .
- FDs are based on domain knowledge , intrinsic features specific to your data.
 - Database engines cannot identify FDs for you; the designer must specify them.

Example: Functional Dependencies

Relation: Students(name, dorm, courseTaken, dept, favCourse)

- Reasonable FDs to assert:
 - $\text{name} \rightarrow \text{dorm}, \text{favCourse}$ (a student has one dorm and one favourite course)
 - Equivalent to: $\text{name} \rightarrow \text{dorm}$ and $\text{name} \rightarrow \text{favCourse}$
 - $\text{courseTaken} \rightarrow \text{dept}$ (each course is offered by one department)

These FDs show the only key is {name, courseTaken}.

Example: Functional Dependencies

- **FDs are a generalization of keys:**
 - Superkey $X \rightarrow R$ means X determines ALL attributes.
 - An FD can involve just a subset of the key.

Properties of FDs

1. Trivial FDs: right side is a subset of the left side.

- $A \rightarrow A$ and $ABC \rightarrow A$ always hold (trivial, not useful).
- $ABC \rightarrow AD$ simplifies to $ABC \rightarrow D$ (remove trivial part). This is "singleton form".

2. Splitting / Combining

- $X \rightarrow A_1A_2\dots A_n$ holds iff each $X \rightarrow A_i$ holds (split the right side freely).
- $A \rightarrow BC$ is equivalent to $A \rightarrow B$ and $A \rightarrow C$.
- There is NO splitting rule for the left side.

Properties of FDs

3. Armstrong's Axioms

- **Reflexivity:** if $X \supseteq Y$ then $X \rightarrow Y$.
- **Augmentation:** if $X \rightarrow Y$ then $XZ \rightarrow YZ$ for any Z .
- **Transitivity:** if $X \rightarrow Y$ and $Y \rightarrow Z$ then $X \rightarrow Z$.
- **Union / Decomposition** follow from the above.

How do you identify FDs?

- FDs are based on domain knowledge
 - Intrinsic features of the data that is specific to your use case.
 - Something you know (or assume) about the data.
- Database engines cannot identify FDs for you
 - Designer must specify them as part of the schema.
 - DBMS can only enforce FDs when told to do so.
- DBMSs cannot “optimize” FDs
 - It has only a finite sample of the data.
 - A FD constrains the entire domain.

Algorithm: Attribute Closure (Y^+)

Closure Y^+ relative to FD set F

The set of all attributes that can be derived from Y using the FDs in F .

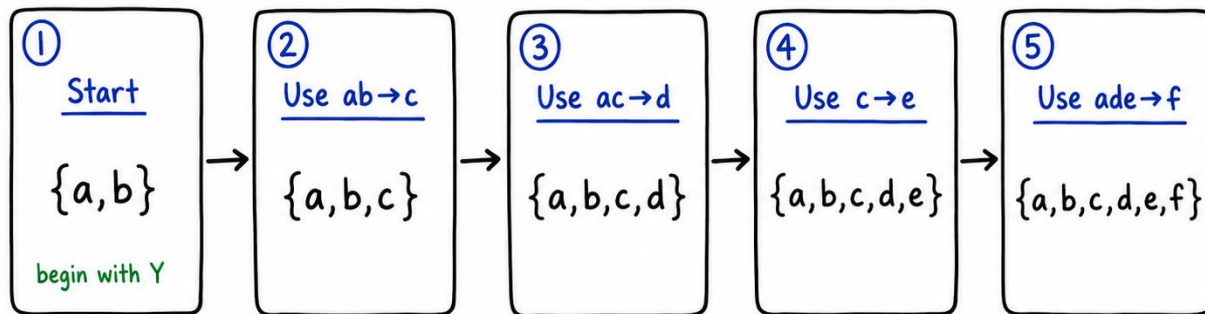
- Starting from a set of attributes Y , what else can we figure out using our FDs?
- **Step 1:** Write down your starting attributes. That is your current set.
- **Step 2:** Look through every FD. If you already have everything on the left side, add the right side to your set.
- **Step 3:** Keep repeating Step 2 until nothing new can be added.

Algorithm: Attribute Closure (Y^+)

Example: find $\{a,b\}^+$

$R(a,b,c,d,e,f)$

FDs: $ab \rightarrow c$, $ac \rightarrow d$, $c \rightarrow e$, $ade \rightarrow f$



Keep adding attributes you can infer until nothing new can be added.

$\{a,b\}^+ = \{a,b,c,d,e,f\}$

all attributes reached

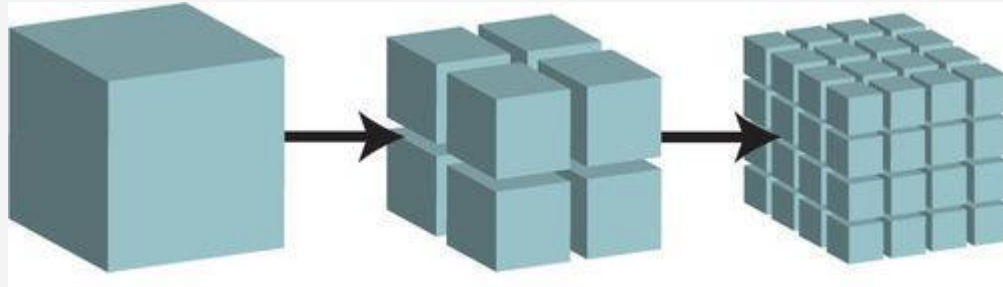
so ab is a superkey
and here it is a candidate key

Handout Time!

Part II

Schema

Decomposition



Relational Schema Design

- Goal is to avoid redundancy and the anomalies it enables.
 - **Update anomaly**: one occurrence of a fact is changed, but not all occurrences have been updated.
 - **Deletion anomaly**: valid fact is lost when a tuple is deleted.

name	dorm	courseTaken	dept	favCourse
Janeway	Voyager	Databases	CS	Ethics
Janeway	???	Ethics	Philosophy	???
Spock	Enterprise	Databases	???	Databases

Data is redundant, because the ??? can be figured out by using the FDs.

Goal of Decomposition

- **Eliminate redundancy by decomposing a relation into several smaller relations.**

Goal of Decomposition

- Eliminate redundancy by decomposing a relation into several smaller relations.
- **Check that a decomposition does not lead to "bad design".**

Goal of Decomposition

- Eliminate redundancy by decomposing a relation into several smaller relations.
- Check that a decomposition does not lead to "bad design".
- **Good decomposition must identify how to split F into subsets applying to each new relation.**

Goal of Decomposition

- Eliminate redundancy by decomposing a relation into several smaller relations.
- Check that a decomposition does not lead to "bad design".
- Good decomposition must identify how to split F into subsets applying to each new relation.
- **Compute the projection of FDs onto the new relations to verify.**

Goal of Decomposition

- Eliminate redundancy by decomposing a relation into several smaller relations.
- Check that a decomposition does not lead to "bad design".
- Good decomposition must identify how to split F into subsets applying to each new relation.
- Compute the projection of FDs onto the new relations to verify.
- **Potential losses when decomposing:**
 - **FD Loss:** attribute A and B land in different relations; enforcing $A \rightarrow B$ requires a costly join.
 - **Join Loss:** joining the pieces produces extra (spurious) tuples.

Goal of Decomposition

- Eliminate redundancy by decomposing a relation into several smaller relations.
- Check that a decomposition does not lead to "bad design".
- Good decomposition must identify how to split F into subsets applying to each new relation.
- Compute the projection of FDs onto the new relations to verify.
- Potential losses when decomposing:
 - FD Loss: attribute A and B land in different relations; enforcing $A \rightarrow B$ requires a costly join.
 - Join Loss: joining the pieces produces extra (spurious) tuples.
- **Goal: minimize anomalies without losing information.**

Goal of Decomposition

- Eliminate redundancy by decomposing a relation into several smaller relations.
- Check that a decomposition does not lead to "bad design".
- Good decomposition must identify how to split F into subsets applying to each new relation.
- Compute the projection of FDs onto the new relations to verify.
- Potential losses when decomposing:
 - FD Loss: attribute A and B land in different relations; enforcing $A \rightarrow B$ requires a costly join.
 - Join Loss: joining the pieces produces extra (spurious) tuples.
- **Goal:** minimize anomalies without losing information.

Schema Decomposition

Given relation R and FDs F

- Split R into R_i s.t. for all i $R_i \subset R$ (no new attributes)
- Split F into F_i s.t. for all i , F entails F_i (no new FDs)
- F_i involves only attributes in R_i

Caveat: entirely possible to lose information

- F^+ may entail FD f which is not in $(\bigcup_i F_i)^+$
- => Decomposition lost some FDs
- Possible to have $R \subset \bowtie_i R_i$
- => Decomposition lost some relationships

Goal: minimize anomalies without losing info

An issue with decomposition is **information loss**. We may not be able to reconstruct the corresponding instance of the original relation.

Good Properties of Decomposition

- 1. Lossless Join Decomposition
 - When we join decomposed relations we should get exactly what we started with.
- 2. Avoid Anomalies
 - Avoid redundant data.
- 3. Dependency Preservation
 - $(F_1 \cup \dots \cup F_n)^+ = F^+$

Example: Splitting Relations

Student Name	Student Email	Course	Instructor
Alice	alice@utoronto.ca	CSC 343	Liut
Alice	alice@utoronto.ca	CSC 209	Petersen
Bob	bob@utoronto.ca	CSC 148	Zingaro
Laura	laura@utoronto.c	CSC 343	Dema

Students (email, name)

Courses (code, instructor)

Taking (email, courseCode)

Students $\bowtie$ Taking $\bowtie$ Courses has additional tuples!

- (Alice, alice@utoronto.ca, CSC 343, Jones), but Alice is not in Dema's section of CSC 343
- (Laura, laura@utoronto.ca, CSC 343, Liut), but Laura is not in Liut's section of CSC 343

Why did this happen? How to prevent it?

Good Properties of Decomposition

1. Lossless Join

Joining the decomposed relations gives back exactly the original relation.

$$r = r_1 \bowtie r_2 \bowtie \dots \bowtie r_k \quad \text{where } r_i = \Pi R_i(r)$$

Lossy decompositions yield MORE tuples than they should.

2 & 3. Anomaly-Free & Dependency Preservation

Anomaly-Free: no redundant data, no update / deletion anomalies.

Dependency Preservation: the union of the projected FD sets is equivalent to the original FD set:

$$(F_1 \cup \dots \cup F_k)^+ = F^+$$

Testing for Losslessness

- A (binary) decomposition of $R = (R, F)$ into $R_1 = (R_1, F_1)$ and $R_2 = (R_2, F_2)$ is lossless if and only if:
 - either the FD $(R_1 \cap R_2) \rightarrow R_1$ is in F^+ ; OR
 - the FD $(R_1 \cap R_2) \rightarrow R_2$ is in F^+ .
- all attributes common to both R_1 and R_2 functionally determine ALL the attributes in R_1 ; OR
- all attributes common to both R_1 and R_2 functionally determine ALL the attributes in R_2 .

TLDR: A decomposition is lossless if the two tables share at least one column, and that shared column is enough to uniquely identify all the other columns in at least one of the tables, i.e. it's a key in at least one of them.

Lending(branch-name, branch-city, assets, customer-name, loan-number, amount)

Lending

branch-name	branch-city	assets	customer-name	loan-number	amount
Downtown	Metropolis	1200000	Alice Smith	L-1001	5000
Downtown	Metropolis	1200000	Bob Johnson	L-1002	7000
Uptown	Gotham	800000	Carol White	L-2001	4000

Branch = R_1

branch-name	branch-city	assets
Downtown	Metropolis	1200000
Uptown	Gotham	800000

Common attributes:

$$R_1 \cap R_2 = \{\text{branch-name}\}$$

LoanInfo = R_2

branch-name	customer-name	loan-number	amount
Downtown	Alice Smith	L-1001	5000
Downtown	Bob Johnson	L-1002	7000
Uptown	Carol White	L-2001	4000

★ RULE (simple!)

Lossless if the common attributes determine all attributes of one piece.

branch-name $\rightarrow$ branch-city, assets

So $\{\text{branch-name}\}$ determines all of Branch (R_1).



Therefore the decomposition is

LOSSLESS

Joining on branch-name recreates the original information (no spurious tuples).



If $(R_1 \cap R_2)$ is a superkey of R_1 or R_2 , the decomposition is lossless.

Projecting FDs onto a Sub-relation

- Once we split a relation, we re-factor the FDs so each involves only attributes of the new relation.
- FD Projection Algorithm for sub-relation $R_i \subset R$:
 - Start with $F_i = \emptyset$.
 - For each subset X of R_i : compute X^+ using F . For each $A \in X^+$ that is also in R_i , add $X \rightarrow A$ to F_i .
 - Compute the minimal basis of F_i .
- **Efficiency tips:** ignore trivial FDs, trivial subsets ($\emptyset$ or all attrs), and supersets of X if $X^+ = R$.
- Even with these tricks, projection is expensive, exponential in the number of attributes.

Minimal Basis (Minimal Cover)

- A minimal basis F' of F satisfies:
 - Right sides are singletons.
 - No FD can be removed (every FD is necessary).
 - No attribute can be removed from any left side.

Minimal Basis (Minimal Cover)

- Construction steps:
 - a. Split all RHS into singletons.
 - b. For each FD f , try removing it. If the remaining set still entails f , remove it.
 - c. For each attribute i on the LHS of each FD, try removing i — if the result still entails the same FD, remove it.
 - d. Repeat steps 2–3 until no more progress.

Minimal Basis (Minimal Cover)

Minimal Basis

- Example:
- $F = \{A \rightarrow AC, B \rightarrow ABC, D \rightarrow ABC\} \rightarrow \text{Minimal Basis} = \{A \rightarrow C, B \rightarrow A, D \rightarrow B\}.$

Minimal Basis (Minimal Cover)

Minimal Basis

- Example:
- $F = \{A \rightarrow AC, B \rightarrow ABC, D \rightarrow ABC\}$

Minimal Basis (Minimal Cover)

Minimal Basis

- Example:
- $F = \{A \rightarrow AC, B \rightarrow ABC, D \rightarrow ABC\} \rightarrow \text{Minimal Basis} = \{A \rightarrow C, B \rightarrow A, D \rightarrow B\}.$

Original relation: $R(A, B, C, D)$

FDs: $A \rightarrow B, B \rightarrow C, C \rightarrow D$

Project onto $R_i(A, C, D)$

① Try $X = \{A\}$

$A^+ = \{A, B, C, D\}$

Inside R_i : $\{A, C, D\}$

Add $A \rightarrow C, A \rightarrow D$

② Try $X = \{C\}$

$C^+ = \{C, D\}$

Add $C \rightarrow D$

③ Try $X = \{D\}$

$D^+ = \{D\}$

Add nothing



Ignore trivial FDs

~~$A \rightarrow A$~~
 ~~$D \rightarrow D$~~

Projected FDs on R_i : $A \rightarrow C, A \rightarrow D, C \rightarrow D$

Minimal basis: $A \rightarrow C, C \rightarrow D$ ~~$A \rightarrow D$~~ ← redundant



Key idea:

Projection = find all dependencies that hold using only attributes of the new relation.



Need to check many subsets of R_i

→ can be expensive.

Original relation: $R(A, B, C, D)$

FDs: $A \rightarrow B, B \rightarrow C, C \rightarrow D$

Project onto $R_i(A, C, D)$

① Try $X = \{A\}$

$A^+ = \{A, B, C, D\}$

Inside R_i : $\{A, C, D\}$

Add $A \rightarrow C, A \rightarrow D$

② Try $X = \{C\}$

$C^+ = \{C, D\}$

Add $C \rightarrow D$

③ Try $X = \{D\}$

$D^+ = \{D\}$

Add nothing



Ignore trivial FDs

~~$A \rightarrow A$~~
 ~~$D \rightarrow D$~~

Projected FDs on R_i : $A \rightarrow C, A \rightarrow D, C \rightarrow D$

Minimal basis: $A \rightarrow C, C \rightarrow D$ ~~$A \rightarrow D$~~ ← redundant



Key idea:

Projection = find all dependencies that hold using only attributes of the new relation.



Need to check many subsets of R_i
→ can be expensive.

Handout Time!

Part III

Normal Forms



Motivation for Normal Forms

- Normal forms give us a systematic way to assess and improve a schema.
- "Normal" = conforming to a standard that guarantees good properties (no anomalies).
- Hierarchy of normal forms (strictest to least strict):
 - $BCNF \subseteq 3NF \subseteq 2NF \subseteq 1NF$
 - 1NF - no multi-valued attributes (no lists stored in a cell).
 - 2NF - non-prime attributes depend on the whole candidate key.
 - 3NF - non-prime attributes depend only on superkeys (or RHS is prime).
 - BCNF - every non-trivial FD has a superkey on the LHS.
- We focus on BCNF and 3NF in this course.

1st Normal Form (1NF)

1NF

- No multi-valued attributes allowed - no storing a list of values in one cell.
- Counter-example: Course(name, instructor, [student, email]*)
 - Alice has two emails → two rows with repeated course/instructor data.
- **Solution:** separate the repeating group into its own relation.
- All standard relational tables must satisfy 1NF by definition.

Name	Instructor	Student Name	Student Email
CSC 343	Liut	Alice	alice@utoronto.ca
CSC 343	Liut	Alice	alice@cs.toronto.edu
CSC 343	Liut	Bob	bob@utoronto.ca
CSC 148	Simion	Laura	laura@utoronto.ca

2nd Normal Form (2NF)

2NF

- Non-prime attributes depend on candidate keys.
 - Consider non-prime attributes A
 - Then there exists a FD $X \rightarrow A$, and X is a candidate key.
- Counterexample
 - Movies(title, year, star, studio, studioAddress, salary)
 - FDs: title, year $\rightarrow$ studio and studio $\rightarrow$ studioAddress and star $\rightarrow$ salary

Title	Year	Star	Studio	StudioAddr	Salary
Star Wars	1977	Hamill	LucasFilm	1 Lucas Way	\$100,000
Star Wars	1977	Ford	LucasFilm	1 Lucas Way	\$100,000
Star Wars	1977	Fisher	LucasFilm	1 Lucas Way	\$100,000
Patriot Games	1992	Ford	Paramount	Cloud 9	\$2,000,000
Last Crusade	1989	Ford	LucasFilm	1 Lucas Way	\$1,000,000

3rd Normal Form (3NF)

3NF

- Non-prime attributes depend **only** on superkeys.
 - Consider FD $X \rightarrow A$
 - Either X is a superkey OR A is **prime** (part of a key)
- Counterexample
 - $\text{studio} \rightarrow \text{studioAddr}$ (studioAddr depends on studio which is not a candidate key)

Title	Year	Studio	StudioAddr
Star Wars	1977	LucasFilm	1 Lucas Way
Patriot Games	1992	Paramount	Cloud 9
Last Crusade	1989	LucasFilm	1 Lucas Way

1. Given relation and FDs

R: $\{ \text{StudentID}, \text{StudentName}, \text{DeptID}, \text{DeptName}, \text{AdvisorID}, \text{AdvisorName} \}$

Functional Dependencies F:

- ① $\text{StudentID} \rightarrow \text{StudentName}, \text{DeptID}$
- ② $\text{DeptID} \rightarrow \text{DeptName}, \text{AdvisorID}$
- ③ $\text{AdvisorID} \rightarrow \text{AdvisorName}$

Original relation R (not in 3NF)

StudentID	StudentName	DeptID	DeptName	AdvisorID	AdvisorName
101	Alice	D1	CS	A1	Dr. Smith
102	Bob	D1	CS	A1	Dr. Smith
103	Carol	D2	Math	A2	Dr. Lee

2. Decompose to 3NF

Create a relation for each FD group.

① $R_1(\text{StudentID}, \text{StudentName}, \text{DeptID})$

StudentID	StudentName	DeptID
101	Alice	D1
102	Bob	D1
103	Carol	D2

② $R_2(\text{DeptID}, \text{DeptName}, \text{AdvisorID})$

DeptID	DeptName	AdvisorID
D1	CS	A1
D2	Math	A2

③ $R_3(\text{AdvisorID}, \text{AdvisorName})$

AdvisorID	AdvisorName
A1	Dr. Smith
A2	Dr. Lee

3. Check key

StudentID is a key of R.

It appears in R_1 , so the decomposition is lossless.

4. Why this is 3NF

- In each relation, the left side of every FD is a key.
- No transitive dependencies.
- Therefore, the decomposition is in 3NF.

Boyce-Codd Normal Form (BCNF)

BCNF

- Stricter than 3NF: for every non-trivial FD $X \rightarrow A$, X must be a superkey.
- Non-trivial means $A \notin X$.
- Counter-example: Drinkers(name, addr, beersLiked, manf, favBeer)
 - Key: {name, beersLiked}
 - $\text{name} \rightarrow \text{addr}, \text{favBeer}$ — name is not a superkey $\rightarrow$ BCNF violation.
 - $\text{beersLiked} \rightarrow \text{manf}$ — beersLiked is not a superkey $\rightarrow$ BCNF violation.
- BCNF Decomposition Algorithm:
 - Find a BCNF violation $X \rightarrow Y$ (X not a superkey).
 - Compute X^+ . Replace R with $R_1 = X^+$ and $R_2 = R - (X^+ - X)$.
 - Recursively decompose R_1 and R_2 , projecting FDs each time.

1. Given relation and FDs

$R(A, B, C)$

Functional Dependencies F:

- 1) $A \rightarrow B$
- 2) $B \rightarrow C$

Original relation $\bar{R}$ (not in BCNF)

A	B	C
a1	b1	c1
a1	b1	c2
a2	b2	c3
a3	b3	c4

Why R is not in BCNF?

$A \rightarrow B$ and $B \rightarrow C$.

A is a key ($A^+ = \{A, B, C\}$),
but B is not a superkey
($B^+ = \{B, C\}$).

Since $B \rightarrow C$ and B is not
a superkey, R violates BCNF.

2. Decompose (remove BCNF violation)

Choose violating FD: $B \rightarrow C$

Decompose R into:

- $R_1(B, C)$
- $R_2(A, B)$

$R_1(B, C)$

B	C
b1	c1
b1	c2
b2	c3
b3	c4

$R_2(A, B)$

A	B
a1	b1
a2	b2
a3	b3

This decomposition is
lossless because

$R_1 \cap R_2 = \{B\}$
and $\{B\} \rightarrow R_1$
($B \rightarrow C$).

3. Check BCNF for each result

$R_1(B, C)$: FDs on $R_1 \rightarrow B \rightarrow C$

B is a key for R_1 ($B^+ = \{B, C\}$).

So R_1 is in BCNF.

$R_2(A, B)$: FDs on $R_2 \rightarrow A \rightarrow B$

A is a key for R_2 ($A^+ = \{A, B\}$).

So R_2 is in BCNF.

4. Result

BCNF decomposition of $R(A, B, C)$:

$R_1(B, C)$ and $R_2(A, B)$

The decomposition is lossless and
each relation is in BCNF.

BCNF vs. 3NF - Trade-offs

BCNF

- ✓ Lossless join
- ✓ No anomalies (anomaly-free)
- ✗ May NOT preserve all dependencies

BCNF eliminates all redundancy caused by FDs, at the cost of sometimes losing the ability to enforce an FD in a single relation.

3NF

- ✓ Lossless join
- ✓ Dependency preservation
- ✗ May still allow some anomalies

3NF tolerates FDs where the RHS is prime — this flexibility is what saves dependency preservation.

Use domain knowledge to choose between them.